LLM Pre-training 与 Continual Pre-Training (CPT) 深度技术分析

作者: AI Research Analysis | **日期**: 2026年3月 | **版本**: v1.0

本文从算法原理、数据工程、分布式训练、业界前沿案例、实操代码及关键论文六大维度，对大语言模型预训练（Pre-training）与持续预训练（Continual Pre-Training, CPT）进行全面而深入的技术剖析。

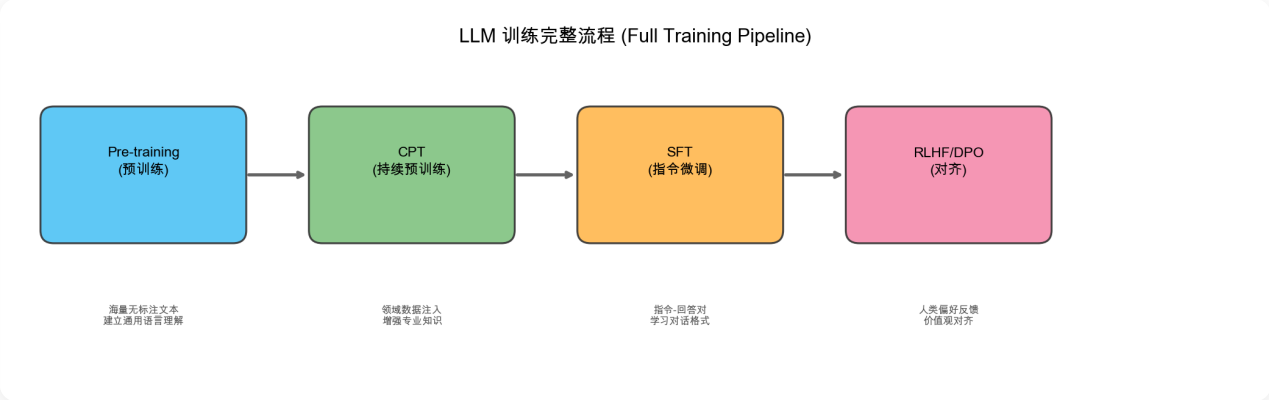
目录

- 一、Pre-training 概述与核心原理
- 二、模型架构演进与关键设计
- 三、训练目标与损失函数
- 四、Scaling Laws：规模法则
- 五、数据工程：采集、清洗与配比
- 六、分布式训练框架与并行策略
- 七、Continual Pre-Training (CPT) 深度解析
- 八、业界前沿案例深度分析
- 九、实操指南与代码示例
- 十、前沿趋势与核心论文索引

一、Pre-training 概述与核心原理

1.1 什么是预训练?

预训练（Pre-training）是大语言模型训练流程的**第一个也是最关键的阶段**。其核心思想是：在**海量无标注文本数据**上，通过自监督学习任务让模型学习语言的通用表征、语法结构、世界知识和推理能力。




```
┌──────────────────────────────────────────────────────────────┐
│                        LLM 训练完整流程                       │
│                                                              │
│  ┌──────────┐    ┌──────────┐    ┌──────────┐    ┌──────────┐  │
│  │Pre-train │──▶│   CPT    │──▶│   SFT    │──▶│ RLHF/    │  │
│  │(预训练)  │    │(持续预训练)│    │(指令微调)│    │  DPO     │  │
│  └──────────┘    └──────────┘    └──────────┘    └──────────┘  │
│       │              │              │              │          │
│  海量无标注      领域数据        标注指令数据     人类偏好反馈     │
│  文本语料        注入知识        学习对话格式     价值对齐        │
└──────────────────────────────────────────────────────────────┘
```

1.2 预训练的数学本质

从数学角度看，预训练本质上是对语言的**概率分布进行建模**。给定 token 序列 $x_1, x_2, \ldots, x_n$，自回归语言模型将联合概率分解为：

$$P(x_1, x_2, \ldots, x_n) = \prod P(x_t \mid x_1, x_2, \ldots, x_{t-1})$$

模型通过最大化训练语料的对数似然来学习参数 θ：

```
L(θ) = −Σ log P_θ(x_t | x_1, ..., x_{t−1})
```

1.3 Pre-training vs CPT vs SFT 对比

维度	Pre-training	CPT	SFT
数据规模	数万亿 Tokens	数百亿~数万亿 Tokens	数十万~数百万样本
数据类型	通用网页/书籍/代码	领域特定+通用混合	指令-回答对
训练目标	Next Token Prediction	Next Token Prediction	Next Token Prediction (on response)
学习率	较高 (1e-4 ~ 5e-4)	中等 (1e-5 ~ 5e-5)	较低 (1e-5 ~ 2e-5)
计算成本	极高 (数百万~数千万美元)	高 (数万~数十万美元)	中等 (数千~数万美元)
核心目标	建立通用语言理解	注入领域知识	学习指令遵从格式

二、模型架构演进与关键设计

2.1 Transformer 架构基础

现代 LLM 几乎全部基于 **Decoder-only Transformer** 架构（GPT 系列开创），其核心组件包括：

```
┌──────────────────────────────────────────┐
│          Decoder-only Transformer         │
│                                           │
│  Input Embeddings + Positional Encoding   │
│           ↓                               │
│  ┌──────────────────────────────────┐     │
│  │     Transformer Block × N layers  │     │
│  │  ┌──────────────────────────┐     │     │
│  │  │ RMSNorm (Pre-Normalization)│     │     │
│  │  │          ↓               │     │     │
│  │  │ Masked Multi-Head Attention│     │     │
│  │  │          ↓               │     │     │
│  │  │ Residual Connection      │     │     │
│  │  │          ↓               │     │     │
│  │  │ RMSNorm                  │     │     │
│  │  │          ↓               │     │     │
│  │  │ SwiGLU FFN               │     │     │
│  │  │          ↓               │     │     │
│  │  │ Residual Connection      │     │     │
│  │  └──────────────────────────┘     │     │
│  └──────────────────────────────────┘     │
│           ↓                               │
│  RMSNorm → Linear → Softmax (Vocab)       │
└──────────────────────────────────────────┘
```

2.2 关键架构设计选择

2.2.1 归一化：RMSNorm + Pre-Normalization

当前主流共识是使用 **RMSNorm** 替代传统 LayerNorm，并采用 **Pre-Normalization**（先归一化，再做注意力/FFN计算）。RMSNorm 去掉了均值中心化操作，计算更高效，训练更稳定。几乎所有现代 LLM（LLaMA、Qwen、DeepSeek 等）都采用此方案。

2.2.2 激活函数：SwiGLU

SwiGLU（Swish-Gated Linear Unit）已成为 FFN 层的标配，其公式为：

```
SwiGLU(x, W₁, W₂, W₃) = (Swish(xW₁) ⊗ xW₂)W₃
```

其中 Swish(x) = x · σ(x)。相比传统 ReLU，SwiGLU 在相同参数量下性能更优。

2.2.3 位置编码：RoPE（旋转位置编码）

RoPE 是当前最广泛使用的位置编码方案，核心优势包括： - **相对位置感知**：注意力分数仅依赖于 token 间的相对位置 - **长度外推能力**：通过调整 base frequency 可扩展到更长序列 - **计算高效**：仅需对 query 和 key 应用旋转矩阵

```
import torch
import torch.nn as nn

class RotaryEmbedding(nn.Module):
    def __init__(self, dim, max_seq_len=8192, base=10000):
        super().__init__()
        inv_freq = 1.0 / (base ** (torch.arange(0, dim, 2).float() / dim))
        self.register_buffer("inv_freq", inv_freq)
        t = torch.arange(max_seq_len).float()
        freqs = torch.einsum("i,j->ij", t, self.inv_freq)
        emb = torch.cat((freqs, freqs), dim=-1)
        self.register_buffer("cos_cached", emb.cos())
        self.register_buffer("sin_cached", emb.sin())

def rotate_half(x):
    x1, x2 = x[..., :x.shape[-1]//2], x[..., x.shape[-1]//2:]
    return torch.cat((-x2, x1), dim=-1)

def apply_rotary_pos_emb(q, k, cos, sin):
    q_embed = (q * cos) + (rotate_half(q) * sin)
    k_embed = (k * cos) + (rotate_half(k) * sin)
    return q_embed, k_embed
```

2.2.4 注意力机制演进

方案	代表模型	核心思想	KV Cache 大小
MHA (Multi-Head Attention)	GPT-3	标准多头注意力	100%

方案	代表模型	核心思想	KV Cache 大小
MQA (Multi-Query Attention)	PaLM, Falcon	所有头共享 K,V	~1/n_heads
GQA (Grouped-Query Attention)	LLaMA 2/3, Qwen2.5	分组共享 K,V	~1/n_groups
MLA (Multi-head Latent Attention)	DeepSeek-V2/V3	低秩压缩 K,V	极小(~5%)

MLA（DeepSeek 创新）核心原理：通过低秩压缩将 Key-Value 投影到一个极低维的潜在空间 c_kv，推理时仅需缓存 c_kv 而非完整的 K,V。压缩维度 512 vs 标准维度数千，在几乎不损失性能的情况下大幅降低推理显存。

```
Input Hidden → Down-Projection → c_kv(压缩向量) → Up-Proj K / Up-Proj V
                                                        ↘ 注意力计算
Input Hidden → Down-Projection → c_q(压缩向量)  → Up-Proj Q ↗
```

2.3 Tokenizer 设计

方法	代表	词表大小	特点
BPE	GPT-2/3/4, LLaMA	32K~128K	基于频率合并，最广泛使用
SentencePiece	T5, mT5	32K~256K	基于概率模型，多语言友好
Byte-level BPE	LLaMA 3, GPT-4	128K+	UTF-8 字节级别，覆盖所有语言
BBPE	Qwen2.5	151K	字节级别，对中文更友好

三、训练目标与损失函数

3.1 标准 Next Token Prediction (NTP)

这是几乎所有 Decoder-only LLM 的核心训练目标——模型输出 vocabulary 上的概率分布，使用**交叉熵损失**与真实 token 计算损失。

3.2 Multi-Token Prediction (MTP) — DeepSeek-V3 创新

```
标准 NTP:  x₁ x₂ x₃ → 预测 x₄
MTP:       x₁ x₂ x₃ → 同时预测 x₄, x₅ (通过额外预测头)

优势:
- 更密集的训练信号，加速学习
- 训练时每个位置预测 D=1 个额外 Token
- 推理时可用于 Speculative Decoding，实现约 1.8x 解码加速
```

3.3 Reinforcement Pre-Training (RPT) — 2025年新范式

字节跳动在2025年提出的革命性预训练范式，将 NTP 重新定义为 **Next-Token Reasoning** 任务：

```
传统 NTP:
  Context: "The capital of France is"
  Predict:  → "Paris"

RPT (Next-Token Reasoning):
  Context: "The capital of France is"
  Reasoning: <think> France is a country in Europe.
             Its capital city is known worldwide...
             The answer should be Paris. </think>
  Predict:  → "Paris"

Reward: r = match(predicted_token, ground_truth_token)
```

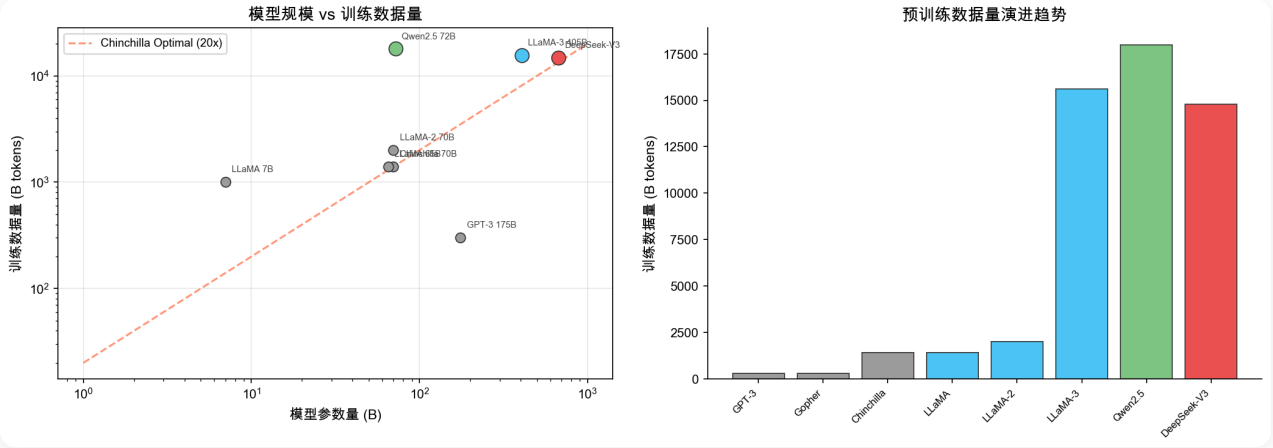
核心创新：将大量未标注文本转化为大规模 RL 数据集，无需外部标注或特定领域奖励函数，显著提升推理能力。

四、Scaling Laws：规模法则

4.1 Chinchilla Scaling Law (DeepMind, 2022)

提出了**计算最优（Compute-Optimal）训练策略**：每个参数约需 **20 个 token** 进行训练。

模型大小	Chinchilla 最优数据量	实际使用数据量（业界趋势）
7B	140B tokens	1T~2T tokens
13B	260B tokens	2T~5T tokens
70B	1.4T tokens	15T~18T tokens
405B	8.1T tokens	15.6T tokens



4.2 Over-training 趋势

现代实践中，业界普遍采用**Over-training**策略（用远超 Chinchilla 最优量的数据训练较小模型），原因在于：**推理成本 >> 训练成本**，更小的模型推理更便宜。

```
Loss
  │  \  Chinchilla Optimal
  │   \     \  Over-training (LLaMA风格)
  │    \     \
  │     \     \──────→ 推理成本更低
  │      \
  └──────────────────── Compute
```

4.3 D-CPT Law：CPT 的 Scaling Law

论文 **D-CPT Law (2024)** 专门研究了 CPT 场景下的 Scaling Law，提出了选择通用语料与领域语料**最优混合比例**的系统方法，可以用小规模实验预测大规模训练效果。

五、数据工程：采集、清洗与配比

5.1 数据处理全流程

```
┌──────┐   ┌──────────┐   ┌──────────┐   ┌──────────┐   ┌──────────┐
│ 采集 │──▶│ 质量过滤 │──▶│  去重    │──▶│ 毒性过滤 │──▶│ 数据混合 │
│      │   │          │   │          │   │          │   │ 配比优化 │
│网页爬取│  │语言检测  │   │URL去重   │   │安全分类器 │   │          │
│API获取│   │困惑度过滤 │   │MinHash   │   │PII移除   │   │Domain权重│
│公开数据│  │质量分类器 │   │SimHash   │   │版权过滤  │   │质量加权  │
└──────┘   └──────────┘   └──────────┘   └──────────┘   └──────────┘
```

5.2 主流预训练数据集

数据集	规模	描述	使用模型
Common Crawl	数十 PB	互联网网页爬取	几乎所有 LLM
The Pile	825 GB	22个高质量子集	GPT-NeoX, Pythia

数据集	规模	描述	使用模型
RedPajama	30T tokens	开源复现 LLaMA 数据	RedPajama 系列
Dolma	3T tokens	AI2 开源	OLMo
FineWeb	15T tokens	HuggingFace 高质量	社区模型
StarCoder Data	1T tokens	代码数据	StarCoder, DeepSeek-Coder

5.3 数据质量过滤关键技术

基于规则的过滤

```
def rule_based_filter(text: str) -> bool:
    """基础规则过滤"""
    if len(text) < 100 or len(text) > 100000:
        return False

    # 语言检测 (fasttext)
    lang, score = detect_language(text)
    if score < 0.65:
        return False

    # 特殊字符比例
    alpha_ratio = sum(c.isalpha() for c in text) / len(text)
    if alpha_ratio < 0.5:
        return False

    # 重复行检测
    lines = text.split('\n')
    if len(set(lines)) / max(len(lines), 1) < 0.3:
        return False

    return True
```

基于困惑度的过滤（使用 KenLM）

```
import kenlm

class PerplexityFilter:
    def __init__(self, model_path, low=10, high=1000):
        self.model = kenlm.Model(model_path)
        self.low, self.high = low, high

    def filter(self, text: str) -> bool:
        words = text.split()
        log_score = self.model.score(' '.join(words))
        ppl = 10 ** (-log_score / max(len(words), 1))
        return self.low < ppl < self.high  # 过低=模板文本, 过高=乱码
```

MinHash + LSH 去重

```
from datasketch import MinHash, MinHashLSH

class DocumentDeduplicator:
    def __init__(self, threshold=0.8, num_perm=128):
        self.lsh = MinHashLSH(threshold=threshold, num_perm=num_perm)
        self.num_perm = num_perm

    def is_duplicate(self, doc_id: str, text: str) -> bool:
        m = MinHash(num_perm=self.num_perm)
        words = text.lower().split()
        for i in range(len(words) - 4):
            ngram = ' '.join(words[i:i+5])
            m.update(ngram.encode('utf-8'))

        if self.lsh.query(m):
            return True
        try:
            self.lsh.insert(doc_id, m)
        except ValueError:
            pass
        return False
```

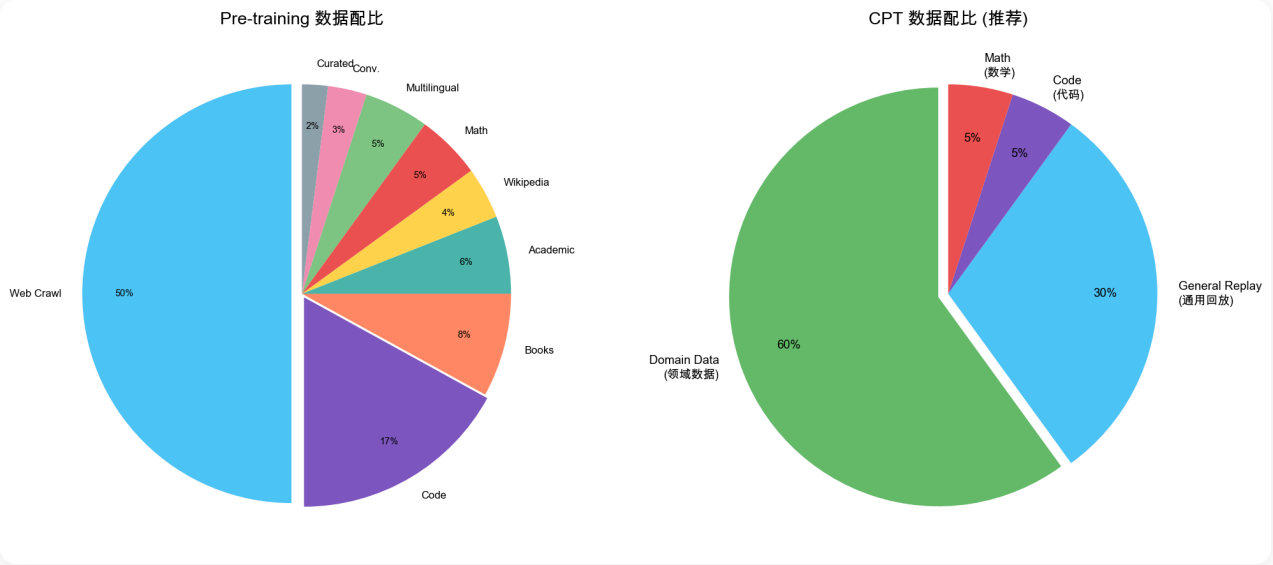
5.4 数据混合策略

SampleMix（美团，2025）— 自下而上范式

传统方法自上而下确定领域权重后域内均匀采样，SampleMix 提出**样本级别**的全局采样：1. K-Means 聚类 → 结构化数据 2. GPT-4o 质量评估 → 样本级质量分 3. 全局跨域采样 → 综合质量+多样性

效果：减少 1.4~2.1x 训练步骤达到基线性能。

典型预训练数据配比参考（LLaMA 3 风格）



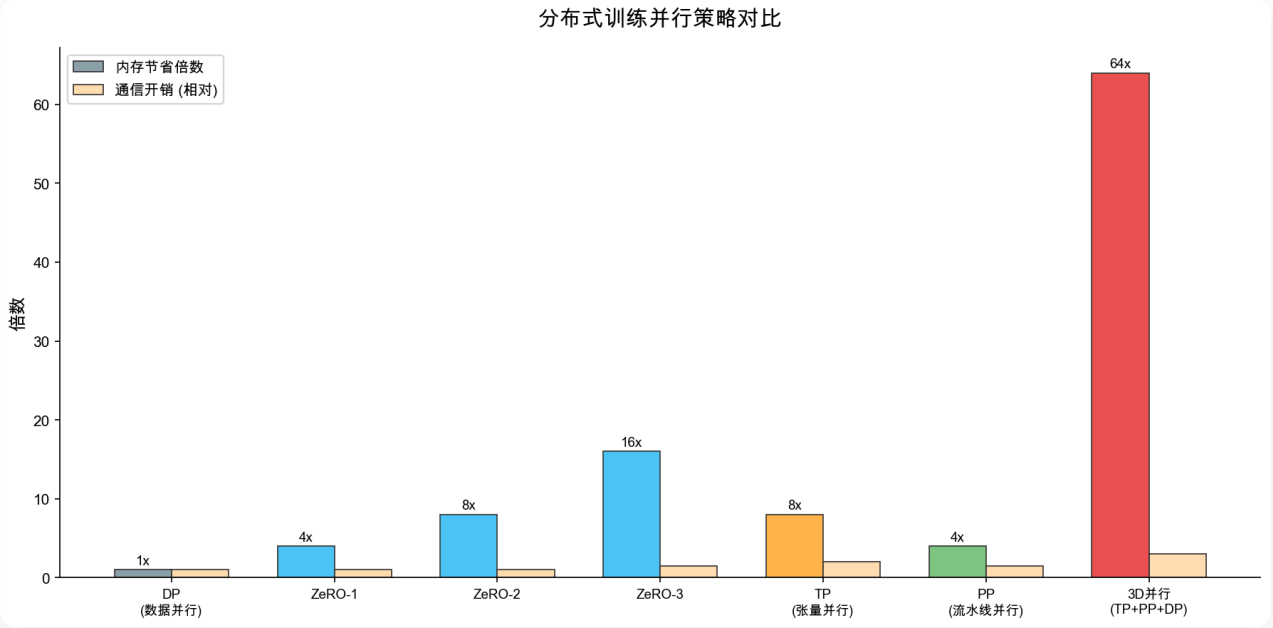

```
DATA_MIXTURE = {
    "web_crawl":        0.50,  # 网页文本（经过严格过滤）
    "code":             0.17,  # 代码数据（GitHub等）
    "books":            0.08,  # 书籍
    "academic_papers":  0.06,  # 学术论文
    "wikipedia":        0.04,  # 百科知识
    "math":             0.05,  # 数学数据
    "multilingual":     0.05,  # 多语言数据
    "conversation":     0.03,  # 对话数据
    "curated_hq":       0.02,  # 精选高质量数据
}
```

六、分布式训练框架与并行策略

6.1 并行策略总览

```
┌──────────────────────────────────────────────────────────────┐
│  数据并行(DP/DDP/FSDP)  │ 每个GPU一份完整/分片模型副本          │
│  张量并行(TP)           │ 切分单层矩阵运算到多GPU               │
│  流水线并行(PP)         │ 切分不同层到不同GPU                   │
│  序列并行(SP)           │ 切分序列维度减少激活值内存            │
│  专家并行(EP)           │ MoE专家分布到不同GPU                  │
│  ZeRO优化               │ 分片optimizer状态/梯度/参数           │
│                                                                 │
│  实际训练: 通常组合 DP + TP + PP + ZeRO                         │
└──────────────────────────────────────────────────────────────┘
```

6.2 ZeRO 优化（DeepSpeed 核心）



阶段	分片内容	内存节省
ZeRO-1	Optimizer States	~4x
ZeRO-2	+ Gradients	~8x

阶段	分片内容	内存节省
ZeRO-3	+ Parameters	~Nx (N=GPU数)

```
// DeepSpeed ZeRO-2 配置示例
{
    "bf16": {"enabled": true},
    "zero_optimization": {
        "stage": 2,
        "allgather_partitions": true,
        "overlap_comm": true,
        "reduce_scatter": true,
        "contiguous_gradients": true
    },
    "gradient_accumulation_steps": "auto",
    "gradient_clipping": 1.0,
    "train_batch_size": "auto",
    "train_micro_batch_size_per_gpu": "auto"
}
```

6.3 FP8 混合精度训练（DeepSeek-V3 创新）

```
前向传播: FP8 (E4M3) — 更高精度
反向传播: FP8 (E5M2) — 更大动态范围
权重存储: BF16
优化器状态: FP32

关键技术: 细粒度量化(tile-wise scaling) + 在线量化(无需离线校准)
效果: 训练速度 ↑ ~40%, 显存 ↓ ~30%, 几乎无精度损失
```

七、Continual Pre-Training (CPT) 深度解析

7.1 定义与动机

持续预训练（CPT） 是在已有预训练模型基础上，使用**领域特定数据**继续预训练以注入新知识或增强特定能力。

核心动机： - 从零预训练成本极高（DeepSeek-V3: $557万，LLaMA 3.1: $6000万） - 领域知识动态更新，无法每次重训 - 通用模型在垂直领域（金融、法律、医疗、工业）表现不足 - CPT 是最具性价比的领域适配方案

7.2 核心挑战：灾难性遗忘

在领域数据上继续训练时，模型会逐渐"遗忘"通用能力，这是 CPT 面临的最大挑战。

7.3 关键技术方案

方案一：数据混合（Data Replay / Mixing）— 最基础也最有效

```
CPT_DATA_MIXTURE = {
    "domain_corpus":    0.60,  # 60% 领域数据（核心）
    "general_replay":   0.30,  # 30% 通用数据回放（防遗忘）
    "code":             0.05,  # 5% 代码（保持代码能力）
    "math":             0.05,  # 5% 数学（保持推理能力）
}
```

方案二：学习率策略 — WSD (Warmup-Stable-Decay)

CPT 的学习率通常为预训练的 1/10~1/20。推荐使用 WSD 策略： - **Warmup 阶段**：快速升温（500~2000 步） - **Stable 阶段**：保持峰值学习率（占 ~70% 训练步数），充分学习领域知识 - **Decay 阶段**：余弦衰减至最低值

```
import math

def wsd_lr_schedule(step, peak_lr=2e-5, min_lr=1e-6,
                    warmup_steps=500, total_steps=50000, stable_ratio=0.7):
    if step < warmup_steps:
        return peak_lr * (step / warmup_steps)

    progress = (step - warmup_steps) / (total_steps - warmup_steps)
    if progress < stable_ratio:
        return peak_lr  # 稳定阶段
```

```
    else:
        decay_progress = (progress - stable_ratio) / (1 - stable_ratio)
        return min_lr + (peak_lr - min_lr) * (1 + math.cos(math.pi * decay_prog
```

方案三：正则化与知识蒸馏

EWC (弹性权重整合)：通过 Fisher 信息矩阵度量参数重要性，对重要参数施加更强的正则化约束。

知识蒸馏：用原始模型作为 Teacher，CPT 模型作为 Student，在领域损失基础上增加 KL 散度约束：

```
# 综合损失 = α * NTP_Loss + (1-α) * KD_Loss
# KD_Loss = KL(student_probs || teacher_probs) * temperature²
total_loss = alpha * ntp_loss + (1 - alpha) * kd_loss
```

方案四：Tokenizer 扩展（多语言/领域适配）

```
from transformers import AutoTokenizer, AutoModelForCausalLM
import torch

def extend_tokenizer_and_embeddings(model_name, new_tokens, output_dir):
    tokenizer = AutoTokenizer.from_pretrained(model_name)
    model = AutoModelForCausalLM.from_pretrained(model_name)

    num_added = tokenizer.add_tokens(new_tokens)
    model.resize_token_embeddings(len(tokenizer))

    # 用已有 token embeddings 的均值初始化新 token
    with torch.no_grad():
        embeddings = model.get_input_embeddings().weight
        mean_emb = embeddings[:-num_added].mean(dim=0)
        for i in range(num_added):
            embeddings[-num_added + i] = mean_emb + torch.randn_like(mean_emb)

    tokenizer.save_pretrained(output_dir)
    model.save_pretrained(output_dir)

# 示例：为金融领域添加专业术语
extend_tokenizer_and_embeddings(
```

```
    "meta-llama/Llama-3.1-8B",
    ["资产负债率", "市盈率", "ROE", "EBITDA", "信用违约互换"],
    "./llama3-finance-extended"
)
```

7.4 D-CPT Law：最优混合比的科学方法

论文 D-CPT Law 提出用小规模实验拟合 Scaling Law，然后预测大规模训练的最优领域数据比例。通过网格搜索多个 (domain_ratio, total_tokens) 组合，拟合幂律函数，即可低成本预测大规模训练效果。

八、业界前沿案例深度分析

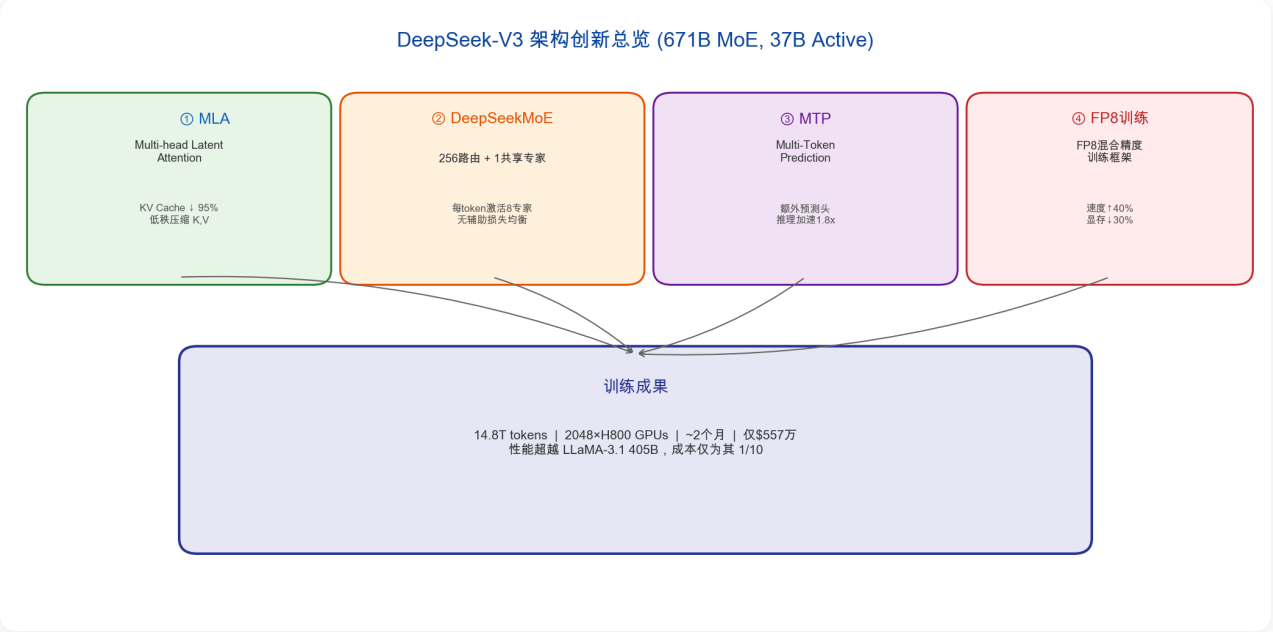
8.1 LLaMA 3 / 3.1 (Meta, 2024)

项目	详情
参数量	8B / 70B / 405B
训练数据	15.6T tokens
架构	Dense Transformer, GQA, RoPE
训练集群	16K H100 GPUs
训练计算量	3.8×10^{25} FLOPs (405B)

训练策略： - Phase 1: 标准预训练，15.6T tokens，序列长度 8192 - Phase 2: 长上下文训练，8K → 128K 分 6 阶段扩展 - Phase 3: 退火（Annealing），使用最高质量数据，上采样数学/代码

数据创新：Duplicated n-gram coverage ratio 去重 + Heuristic + Classifier 双重过滤 + 代码语法检查与编译验证

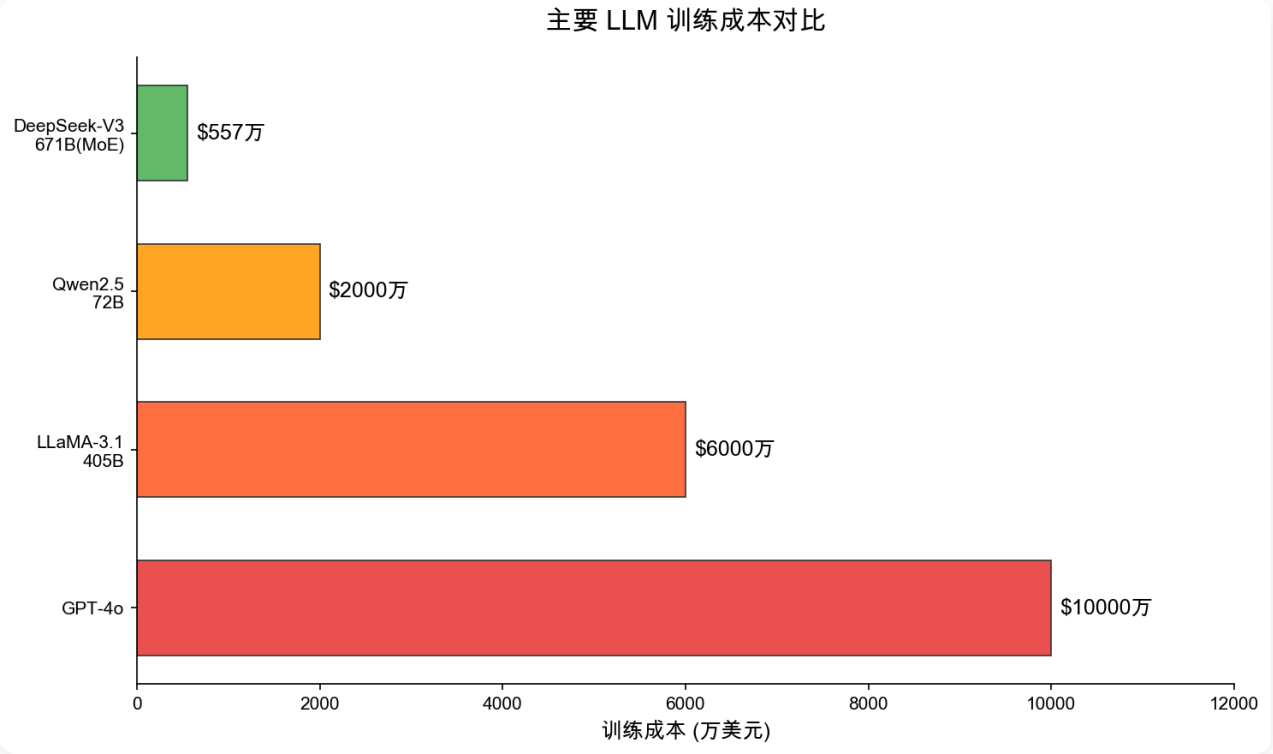
8.2 DeepSeek-V3 (DeepSeek, 2024-2025) ⭐



项目	详情
总参数量	671B (MoE)
激活参数	37B / token
训练数据	14.8T tokens
训练成本	~$557万 (2048 H800)
训练时长	~2个月

四大技术创新： 1. **MLA (Multi-head Latent Attention)**：低秩压缩 KV Cache 至 ~5% 2. **DeepSeekMoE**：256 路由专家 + 1 共享专家，无辅助损失负载均衡 3. **MTP (Multi-Token Prediction)**：更丰富梯度信号 + 1.8x 推理加速 4. **FP8 混合精度**：训练速度 ↑40%，显存 ↓30%

成本对比：



模型	训练成本	GPU集群
GPT-4o	~$1亿	数万H100
LLaMA-3.1 405B	~$6000万	16K H100
DeepSeek-V3	~$557万	2048 H800

8.3 Qwen2.5 (阿里云, 2024-2025)

项目	详情
模型系列	0.5B / 1.5B / 3B / 7B / 14B / 32B / 72B
训练数据	18T tokens（从 Qwen2 的 7T 扩展 2.6x）
后训练	1M+ SFT 样本 + DPO + GRPO

关键改进：预训练数据 7T→18T tokens，后训练引入 GRPO 在线强化学习，长文本生成和结构化数据分析能力显著增强。

8.4 OLMo 2 (AI2, 2025) — 完全开源标杆

项目	详情
参数量	7B / 13B
训练数据	Dolma 3T tokens (完全开源)
开源程度	代码 + 数据 + 检查点 + 训练日志

OLMo 2 是目前最开放的 LLM 训练项目，为研究社区提供了宝贵的可复现基准。

九、实操指南与代码示例

9.1 使用 LLaMA-Factory 进行 CPT

环境准备

```
git clone https://github.com/hiyouga/LLaMA-Factory.git
cd LLaMA-Factory
pip install -e ".[torch,metrics]"
```

CPT 数据格式 (data/domain_corpus.json)

```
[
    {"text": "信用评分模型是金融风控领域的核心工具。FICO评分系统基于五大维度：还款历史(35%)
    {"text": "巴塞尔协议III对银行资本充足率提出了更严格的要求。核心一级资本充足率不得低于4.
]
```

CPT 配置 (configs/cpt_config.yaml)

```
model_name_or_path: meta-llama/Llama-3.1-8B
stage: pt
```

```
do_train: true
finetuning_type: full

dataset: domain_corpus
cutoff_len: 4096
streaming: true

per_device_train_batch_size: 2
gradient_accumulation_steps: 16
learning_rate: 2.0e-5
num_train_epochs: 1.0
lr_scheduler_type: cosine
warmup_ratio: 0.01
weight_decay: 0.1
max_grad_norm: 1.0
bf16: true

output_dir: saves/llama3-8b-finance-cpt
logging_steps: 10
save_steps: 500
deepspeed: configs/ds_z2_config.json
```

启动训练

```
# 单 GPU
llamafactory-cli train configs/cpt_config.yaml

# 多 GPU (8卡)
FORCE_TORCHRUN=1 NNODES=1 NPROC_PER_NODE=8 \
    llamafactory-cli train configs/cpt_config.yaml
```

9.2 原生 PyTorch CPT 核心代码

```
"""原生 PyTorch CPT 训练核心逻辑"""
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer, get_cosine_schedu

def train_cpt(model_name, train_dataloader, output_dir,
              lr=2e-5, warmup_steps=500, max_steps=50000, grad_accum=16):
```

```
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
    model_name, torch_dtype=torch.bfloat16, attn_implementation="flash_atte
).cuda()
model.gradient_checkpointing_enable()

optimizer = torch.optim.AdamW(model.parameters(), lr=lr, betas=(0.9, 0.95),
scheduler = get_cosine_schedule_with_warmup(optimizer, warmup_steps, max_st

global_step = 0
model.train()
optimizer.zero_grad()

for batch in train_dataloader:
    if global_step >= max_steps:
        break

    with torch.cuda.amp.autocast(dtype=torch.bfloat16):
        outputs = model(
            input_ids=batch['input_ids'].cuda(),
            attention_mask=batch['attention_mask'].cuda(),
            labels=batch['labels'].cuda(),
        )
        loss = outputs.loss / grad_accum

    loss.backward()

    if (global_step + 1) % grad_accum == 0:
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()
        scheduler.step()
        optimizer.zero_grad()

    global_step += 1
    if global_step % 10 == 0:
        print(f"Step {global_step} | Loss: {loss.item() * grad_accum:.4f} |

model.save_pretrained(output_dir)
tokenizer.save_pretrained(output_dir)
```

9.3 Megatron-LM 大规模预训练启动脚本

```
#!/bin/bash
# 7B 模型预训练配置示例（32 GPUs = 4 nodes × 8 GPUs）

GPT_ARGS="
    --num-layers 32
    --hidden-size 4096
    --num-attention-heads 32
    --num-key-value-heads 8
    --seq-length 4096
    --micro-batch-size 1
    --global-batch-size 256
    --lr 3e-4
    --min-lr 3e-5
    --train-iters 250000
    --lr-decay-style cosine
    --lr-warmup-iters 2000
    --weight-decay 0.1
    --clip-grad 1.0
    --bf16
    --use-flash-attn-v2
    --use-rotary-position-embeddings
    --normalization RMSNorm
    --swiglu
    --tensor-model-parallel-size 4
    --pipeline-model-parallel-size 2
    --sequence-parallel
    --use-distributed-optimizer
"

torchrun --nproc_per_node 8 --nnodes 4 pretrain_gpt.py $GPT_ARGS
```

9.4 CPT 效果评估

```
"""CPT 训练效果评估"""
import torch, numpy as np
from transformers import AutoModelForCausalLM, AutoTokenizer

def compute_perplexity(model_path, texts, max_length=2048):
```

```
    tokenizer = AutoTokenizer.from_pretrained(model_path)
    model = AutoModelForCausalLM.from_pretrained(
        model_path, torch_dtype=torch.bfloat16).cuda().eval()

    total_loss, total_tokens = 0, 0
    for text in texts:
        inputs = tokenizer(text, return_tensors="pt", max_length=max_length, tr
        with torch.no_grad():
            outputs = model(**inputs, labels=inputs["input_ids"])
            total_loss += outputs.loss.item() * inputs["input_ids"].shape[1]
            total_tokens += inputs["input_ids"].shape[1]
    return np.exp(total_loss / total_tokens)

# 对比评估
domain_texts = ["信用风险度量包括违约概率PD、违约损失率LGD和违约风险暴露EAD...",
                "期权定价中Black-Scholes模型假设标的资产价格服从几何布朗运动..."]

base_ppl = compute_perplexity("meta-llama/Llama-3.1-8B", domain_texts)
cpt_ppl = compute_perplexity("./finance_cpt_output", domain_texts)
print(f"Domain PPL — Base: {base_ppl:.2f}, CPT: {cpt_ppl:.2f} (↓{(base_ppl-cpt_
```

十、前沿趋势与核心论文索引

10.1 预训练新范式

范式	核心思想	代表论文/项目
Reinforcement Pre-Training (RPT)	将 NTP 重构为推理任务 + RL	ByteDance, 2025
Multi-Token Prediction	每步预测多个 token	DeepSeek-V3, Meta 2024
Test-Time Training	推理阶段在线适配	Stanford, 2024
Mixture of Experts (MoE)	稀疏激活大模型	DeepSeek-V3, Switch Transformer

范式	核心思想	代表论文/项目
Over-training / Over-Chinchilla	用更多数据训练更小模型	LLaMA 系列

10.2 核心论文索引

预训练基础

1. **Attention Is All You Need** (Vaswani et al., 2017) — Transformer 原始论文
2. **Language Models are Unsupervised Multitask Learners** (GPT-2, Radford et al., 2019)
3. **Language Models are Few-Shot Learners** (GPT-3, Brown et al., 2020)
4. **LLaMA: Open and Efficient Foundation Language Models** (Touvron et al., 2023)
5. **The Llama 3 Herd of Models** (Meta, 2024) — LLaMA 3 技术报告

Scaling Laws

1. **Scaling Laws for Neural Language Models** (Kaplan et al., 2020)
2. **Training Compute-Optimal Large Language Models** (Chinchilla, Hoffmann et al., 2022)
3. **D-CPT Law: Domain-specific Continual Pre-Training Scaling Law** (Que et al., 2024, arXiv:2406.01375)

架构创新

1. **DeepSeek-V3 Technical Report** (DeepSeek-AI, 2024)
2. **Qwen2.5 Technical Report** (Alibaba, 2024)
3. **RoFormer: Enhanced Transformer with Rotary Position Embedding** (Su et al., 2021)
4. **GQA: Training Generalized Multi-Query Transformer Models** (Ainslie et al., 2023)

CPT 与持续学习

1. **ERNIE 2.0: A Continual Pre-Training Framework** (Sun et al., 2020)

2. **Breaking Language Barriers: Cross-Lingual CPT at Scale** (Zheng et al., 2024, arXiv:2407.02118)
3. **Balancing Continuous Pre-Training and Instruction Fine-Tuning** (Jindal et al., 2024, arXiv:2410.10739)
4. **Construction of Domain-specified Japanese LLM through CPT** (Hirano et al., 2024, arXiv:2404.10555)

数据工程

1. **Programming Every Example: Lifting Pre-training Data Quality Like Experts at Scale** (Zhou et al., 2024, arXiv:2409.17115)
2. **SampleMix: A Sample-wise Pre-training Data Mixing Strategy** (美团, 2025)
3. **The Pile: An 800GB Dataset of Diverse Text** (Gao et al., 2020)
4. **FineWeb: Decanting the Web for the Finest Text Data** (HuggingFace, 2024)

分布式训练

1. **Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism** (NVIDIA, 2020)
2. **ZeRO: Memory Optimizations Toward Training Trillion Parameter Models** (Rajbhandari et al., 2020)
3. **Variance Control via Weight Rescaling in LLM Pre-training** (Owen et al., 2025, arXiv:2503.17500)

新范式

1. **Reinforcement Pre-Training** (ByteDance, 2025) — 强化预训练
2. **Scaling Laws for Native Multimodal Models** (Shukor et al., 2025) — 多模态 Scaling Laws

---

总结

核心要点回顾

```
┌──────────────────────────────────────────────────────────────────────┐
│                        LLM 预训练与 CPT 全景图                        │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────┐    │
│  │                        Pre-training                          │    │
│  │  架构: Decoder-only + RMSNorm + SwiGLU + RoPE + GQA/MLA│    │
│  │  目标: NTP / MTP / RPT                                       │    │
│  │  数据: 数十TB, 严格过滤+去重+配比优化                    │    │
│  │  训练: 3D并行 + ZeRO + 混合精度 + Flash Attention       │    │
│  │  规模: 遵循 Scaling Laws, 趋向 Over-training             │    │
│  └──────────────────────────────────────────────────────────────┘    │
│                              ↓                                       │
│  ┌──────────────────────────────────────────────────────────────┐    │
│  │                    Continual Pre-Training                    │    │
│  │  数据: 领域数据(60-70%) + 通用回放(30-40%)              │    │
│  │  学习率: 预训练的 1/10~1/20, WSD 调度                    │    │
│  │  防遗忘: 数据混合 + EWC + 知识蒸馏                       │    │
│  │  评估: 领域PPL↓ + 通用能力不退化                         │    │
│  │  Scaling: D-CPT Law 指导最优配比                          │    │
│  └──────────────────────────────────────────────────────────────┘    │
│                                                                      │
│  最先进案例:                                                          │
│  • DeepSeek-V3: MoE+MLA+MTP+FP8, 成本仅$557万                        │
│  • LLaMA 3.1: 15.6T tokens, 三阶段训练                               │
│  • Qwen2.5: 18T tokens, GRPO 对齐                                    │
│  • OLMo 2: 完全开源标杆                                              │
│                                                                      │
│  未来方向:                                                            │
│  • 强化预训练(RPT)重新定义预训练范式                                  │
│  • 多模态预训练改写 Scaling Laws                                      │
│  • 更高效的训练方法持续降低成本                                       │
└──────────────────────────────────────────────────────────────────────┘
```

> 📝 **注**: 本文撰写于2026年3月，LLM 领域发展极为迅速，部分内容可能已有更新进展。建议读者结合最新论文和技术报告进行补充阅读。